

Assignment 6 - DDL: Car Rental Database

Student ID: 324943109

Customer - Yaron Besser, [REDACTED]

Designer - Nevo Alani, [REDACTED]

Reviewer - Ofek Goldstein, [REDACTED]

Part 1 - Modeling

1. Customer

Customer:

I run a car rental company with a fleet of over 100 vehicles. I need a database to manage my operations - tracking my cars, storing customer details, recording rentals and payments, and managing fleet maintenance including both past services and upcoming scheduled jobs.

Requirements

- Store all vehicles: license plate, type, manufacturer, size, and daily rate - the foundation of the business, no rental can be processed without a vehicle registry.
- Store customer information: name, address, phone, and email - required for billing, legal documentation, and rental history.
- Record every rental transaction: customer, vehicle, start date, end date, and gross payment - the core financial record linking customers to vehicles.
- Store completed maintenance events per vehicle including service type and dates - critical for fleet health and cost tracking.
- Track upcoming planned maintenance per vehicle - prevents scheduling a vehicle for rental during a service.

2. Design discussion

Designer:

- For billing - do you need us to store a breakdown of the payment, or is the total amount charged per rental enough?
- For maintenance - do you manage past services and upcoming jobs separately, or do you just need to know the current status of each job?
- For the different maintenance jobs - do oil changes, inspections, and other services each have their own pricing and expected duration?

Customer:

- Just the total is enough - my accountant handles the breakdown.
- No need to separate them - I just need to know if a job is done or still coming up.
- Yes - each service type has its own price and how long it takes.

3. Designer confirmation

Designer:

I understand the requirements. The schema consists of five tables, each directly covering one or more of the customer's requirements. No table exists without a clear business justification.

VEHICLES

Covers R1 - fleet management. Each vehicle is a distinct entity, stored once and referenced by transactions and maintenance.

- **license_num** - PK. Unique plate number, natural identifier.
- **vehicle_type, company, size** - Vehicle characteristics for filtering and reporting.
- **daily_rate** - Base rental price per day.

CUSTOMERS

Covers R2 - customer records. Personal details stored once regardless of how many rentals a customer makes.

- **customer_id** - PK. Auto-incremented surrogate key.
- **first_name, last_name** - Full name.
- **city, street** - Address split into two fields for 1NF atomicity.
- **phone, email** - Contact details.

TRANSACTIONS

Covers R3 - rental transactions. Core link between customer and vehicle. Cannot live inside either table without creating redundancy.

- **transaction_id** - PK.
- **customer_id, vehicle_id** - FKs to Customers and Vehicles.
- **start_date, end_date** - Rental period.
- **gross_total** - Total charged. Stored directly - daily rates can change, so we preserve the exact amount at the time of rental.

SERVICE TYPES

Covers R4+R5 - maintenance lookup. Service attributes stored once here - updating a rate changes one row, not every maintenance record (3NF).

- **service_type_id** - PK.
- **type_name** - Name of the service (oil change, inspection...).
- **rate** - Cost of this service type.
- **duration_days** - Expected duration.

MAINTENANCE

Covers R4+R5 - both history and upcoming jobs. One table with a status column replaces two separate tables.

- **service_id** - PK.
- **vehicle_id, service_type_id** - FKs to Vehicles and Service_types.
- **start_date, end_date** - Service dates.
- **status** - 'completed' or 'scheduled'.

Vehicle availability is not stored. A vehicle is available if it has no active transaction covering today's date - always accurate, no risk of inconsistency.

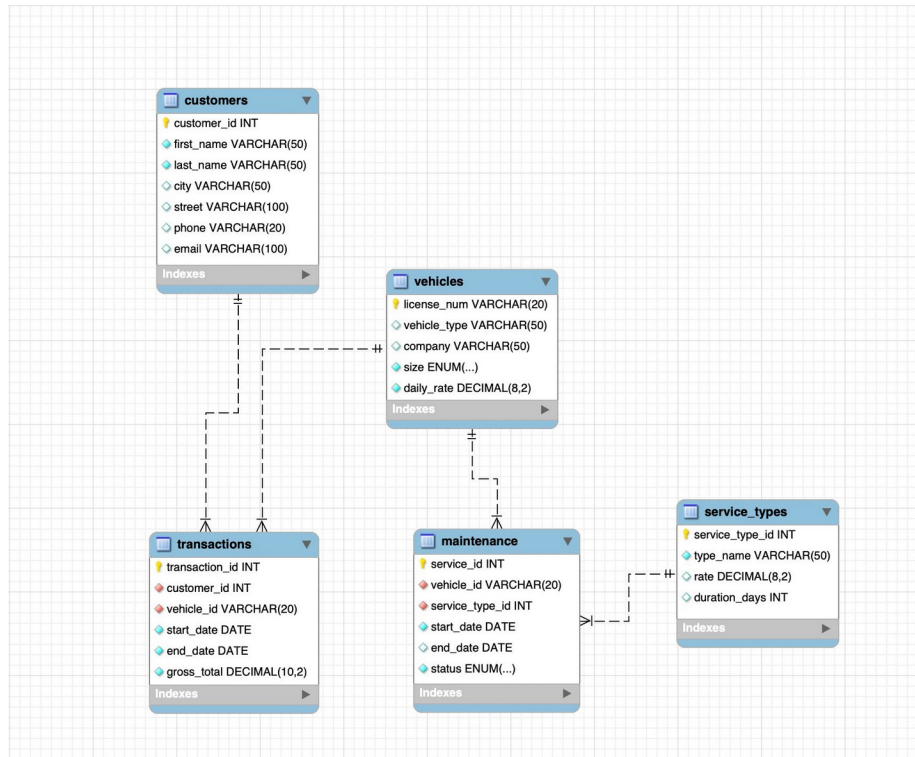
Normalization

- 1NF: All columns are atomic. No repeating groups.
- 2NF: All tables use single-column primary keys - no partial dependencies.
- 3NF: No transitive dependencies. No derived values stored. Service attributes live in service_types, not in every maintenance row.

Changes made during the design process

- **gross_total kept:** Initially considered deriving it from $\text{daily_rate} \times \text{days}$. Decided to store it - daily rates can change and we need to preserve the exact amount charged at the time of each rental.
- **Maintenance tables merged:** Initially two separate tables for history and upcoming jobs. Merged into one with a status column - same structure, no reason to duplicate.
- **Availability flag removed:** Initially considered storing an available field on vehicles. Removed - it is derivable from active transactions and storing it risks going out of sync.

Schema diagram



4. Reviewer

Both the customer and the designer understood the requirements the same way. The customer's five requirements map directly to the five tables in the final schema with no gaps and no extras. Three design decisions - removing derived payment fields, merging maintenance tables, and dropping the availability flag - were raised, discussed, and resolved with clear justification. The final schema is normalized, consistent, and aligned with the customer's stated needs.